

DriftXpress MNIST Results

Experimental record

This report was assembled from the saved code, textual outputs, and embedded figures in the five notebooks in this folder. No notebook was executed or modified while producing this report. The experiment uses MNIST digits $\{0, 8, 2, 4, 7, 9\}$, with 35,788 training examples, 6,005 test examples, batch size 500, latent dimension 32, 200 drift landmarks, and 100 configured training epochs. Because the notebook loops use `range(num_epochs+1)`, the saved runs actually include epochs 0 through 100.

Metric comparability. All five evaluation cells compare decoded generated images against flattened MNIST test pixels. The reported “FID” is the implementation in `src/eval.py`, not canonical Inception-FID; PCA and UMAP are visual projections in these notebooks and do not produce a scalar score.

Aggregate evaluation metrics

Table 1: Standardized pixel-space evaluation. Lower is better for SW, MMD, and FID; higher is better for coverage and entropy.

Algorithm	evaluation space	SW ↓	MMD ↓	FID ↓	coverage ↑	entropy ↑
1	pixels	0.156390	0.142476	32.506738	1.000000	0.999951
2	pixels	0.055698	0.019198	9.749420	1.000000	0.999930
3	pixels	0.145378	0.150643	60.209453	1.000000	0.999966
4	pixels	0.088063	0.043731	15.564459	1.000000	0.999738
5	pixels	0.115568	0.095509	49.368244	1.000000	0.999588

Table 2: Per-digit pixel-space FID values.

Digit	space	Alg. 1	Alg. 2	Alg. 3	Alg. 4	Alg. 5
0	pixel	58.9163	13.2347	86.5164	29.0469	82.2101
8	pixel	52.1156	17.3052	80.4397	24.8819	67.9792
2	pixel	46.5633	19.7732	74.4471	25.1308	66.0384
4	pixel	39.0935	16.1500	66.0835	20.6945	52.5338
7	pixel	34.4265	11.9323	68.3462	9.6142	47.2885
9	pixel	39.9167	14.6916	69.6437	18.5663	51.2729

Algorithm summaries and losses

Algorithm 1: joint conditional training

Algorithm 1 trains the VAE and conditional latent generator in one loop with one optimizer object. Its drift term acts directly on the raw generator latent, so the drift gradient reaches the generator but bypasses the encoder and decoder. The saved run has pixel FID 32.51 and perfect class coverage, although its cache is built before training from sampled encoder latents.

$$\mathcal{L}_1 = \mathcal{L}_{\text{recon}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{d}}\mathcal{L}_{\text{drift}}(z^-) + \lambda_{\text{v}}(\mathcal{L}_{\text{var}}(z^+) + \mathcal{L}_{\text{var}}(z^-)) + \lambda_{\text{c}}(\mathcal{L}_{\text{cov}}(z^+) + \mathcal{L}_{\text{cov}}(z^-)).$$

Algorithm 2: separate VAE and generator optimization

Algorithm 2 first trains the VAE and then freezes it while optimizing a separate conditional latent generator. The generator is trained only with drift, variance, and covariance objectives, so it cannot change the learned decoder or encoder representation. Its pixel-space evaluation gives FID 9.75 with complete class coverage.

$$\begin{aligned}\mathcal{L}_{\text{VAE}} &= \mathcal{L}_{\text{recon}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{v}}\mathcal{L}_{\text{var}}(\mu) + \lambda_{\text{c}}\mathcal{L}_{\text{cov}}(\mu), \\ \mathcal{L}_{\text{G}} &= \lambda_{\text{d}}\mathcal{L}_{\text{drift}}(z^-) + \lambda_{\text{v}}\mathcal{L}_{\text{var}}(z^-) + \lambda_{\text{c}}\mathcal{L}_{\text{cov}}(z^-).\end{aligned}$$

Algorithm 3: fully joint re-encoded drift

Algorithm 3 sends generated latents through the decoder and deterministic encoder mean before applying drift. This creates a differentiable path from the re-encoded drift loss to the generator, decoder, and encoder, without a cycle loss. The saved pixel FID is 60.21, and the generated digit grid shows that the re-encoding path alone does not sufficiently constrain the raw generator latent.

$$\mathcal{L}_3 = \mathcal{L}_{\text{recon}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{d}}\mathcal{L}_{\text{drift}}(\tilde{z}^-) + \lambda_{\text{v}}(\mathcal{L}_{\text{var}}(\mu) + \mathcal{L}_{\text{var}}(\tilde{z}^-)) + \lambda_{\text{c}}(\mathcal{L}_{\text{cov}}(\mu) + \mathcal{L}_{\text{cov}}(\tilde{z}^-)), \quad \tilde{z}^- = \mu_{\phi}(D_{\psi}(G_{\theta}(\varepsilon, c))).$$

Algorithm 4: latent discriminator

Algorithm 4 adds a class-conditional latent discriminator to Algorithm 1. The discriminator is trained with detached real and generated latents, while the generator receives a non-saturating BCE adversarial loss through the temporarily frozen discriminator. Its pixel-space evaluation gives FID 15.56 with complete class coverage.

$$\begin{aligned}\mathcal{L}_4 &= \mathcal{L}_1 + \lambda_{\text{adv}}\mathcal{L}_{\text{adv}}, \\ \mathcal{L}_{\text{disc}} &= \frac{1}{2} [\text{BCE}(D_{\omega}(\text{sg}(z^+), c), 1) + \text{BCE}(D_{\omega}(\text{sg}(z^-), c), 0)], \\ \mathcal{L}_{\text{adv}} &= \text{BCE}(D_{\omega}(z^-, c), 1).\end{aligned}$$

Algorithm 5: pretraining plus raw and re-encoded drift

Algorithm 5 pretrains the VAE for 25 epochs and builds the cache from deterministic encoder means. During fully joint training it averages drift on the raw generator latent and on the decoded–re-encoded latent, preserving both direct generator control and the connected gradient path. The L1 coefficient is present but set to zero, and the saved pixel metrics are SW 0.115568, MMD 0.095509, and FID 49.37.

$$\begin{aligned}\mathcal{L}_{\text{pre}} &= \mathcal{L}_{\text{recon}} + \lambda_{\text{L1}}\mathcal{L}_1 + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}}, \\ \mathcal{L}_5 &= \mathcal{L}_{\text{recon}} + \lambda_{\text{L1}}\mathcal{L}_1 + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{d}}\frac{1}{2} [\mathcal{L}_{\text{drift}}(z^-) + \mathcal{L}_{\text{drift}}(\tilde{z}^-)] \\ &\quad + \lambda_{\text{v}}(\mathcal{L}_{\text{var}}(\mu) + \mathcal{L}_{\text{var}}(\tilde{z}^-)) + \lambda_{\text{c}}(\mathcal{L}_{\text{cov}}(\mu) + \mathcal{L}_{\text{cov}}(\tilde{z}^-)).\end{aligned}$$

Model parameters and optimization settings

The notebook parameter-count cells instantiate constructor defaults rather than the later configured `LATENT_DIM = 32`. The table below gives the parameter counts for the actual trained configuration, recomputed from the architectures used by each training loop.

Saved generated MNIST outputs

Each figure reproduces the embedded notebook output. The rows are labelled by the notebook as real images, reconstructions, and generated samples when all three rows are present.

PCA and UMAP projections

The first projection for each algorithm uses digit labels; the second uses a binary real-versus-generated label. These are visual diagnostics rather than numeric scores. The saved Algorithm 1 figures show an empty UMAP panel, consistent with the previously reported UMAP environment problem; the other notebooks contain rendered PCA/UMAP panels.

Table 3: Architecture and configured optimization parameters.

Algorithm	trainable parameters	architecture	optimization settings	activations
1	357,280	E : 784-192-96-32; D : 32-96-192-784; G : 48-48-48-32; label embedding 16	$T = 2$, $r = 200$, $\lambda_{KL} = 10^{-5}$, $\lambda_d = 30$, $\lambda_v = .1$, $\lambda_c = .3$, AdamW 10^{-3}	LeakyReLU encoder/decoder, SiLU generator, Sigmoid output
2	357,280 total; VAE 348,560, G 8,720	Same VAE and generator as Algorithm 1; VAE frozen in Stage II	$T = 2$, $r = 200$, same loss weights; 100 VAE epochs then 100 generator epochs, AdamW 10^{-3}	LeakyReLU, SiLU, Sigmoid
3	357,280	Same VAE and generator; generated path includes $G \rightarrow D \rightarrow \mu$	$T = 2$, $r = 200$, same loss weights; AdamW 10^{-3}	LeakyReLU, SiLU, Sigmoid
4	357,280 main model + 7,457 discriminator	Algorithm 1 model plus D_ω : 48-64-64-1 with label embedding 16	$T = 2$, $r = 200$, $\lambda_{adv} = 1$, model/discriminator LR 10^{-3}	LeakyReLU discriminator; BCE-with-logits
5	357,280	Same VAE and generator; deterministic μ cache; raw and re-encoded drift	25 VAE pretraining epochs, then 100 joint epochs; $T = 2$, $r = 200$, $\lambda_{L1} = 0$, AdamW 10^{-3}	LeakyReLU, SiLU, Sigmoid

Rows: real MNIST | reconstructions | generated samples conditioned on the row labels

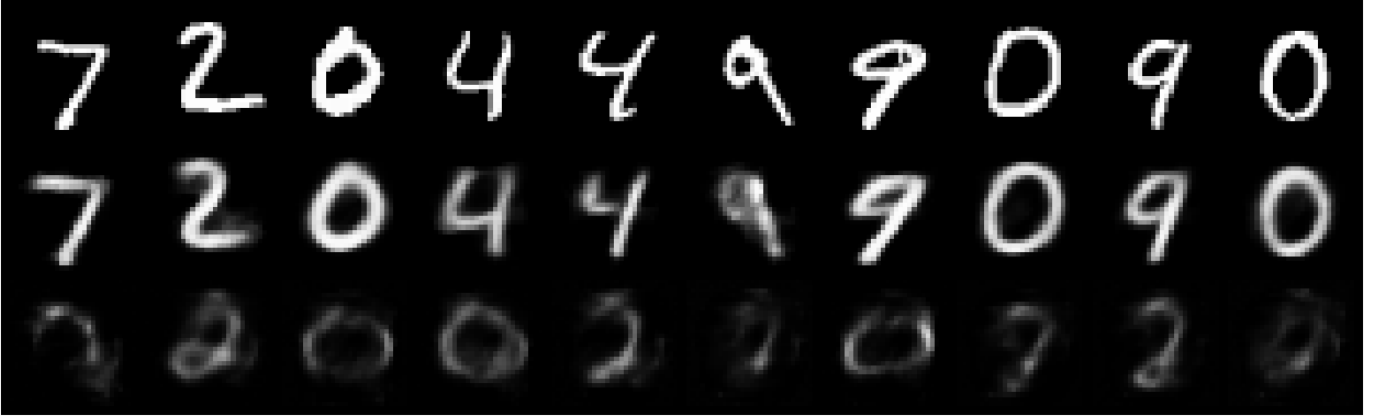


Figure 1: Algorithm 1 saved MNIST output.

Rows: real MNIST | reconstructions | generated samples conditioned on the row labels



Figure 2: Algorithm 2 saved MNIST output.

Rows: real MNIST | reconstructions | generated samples conditioned on the row labels

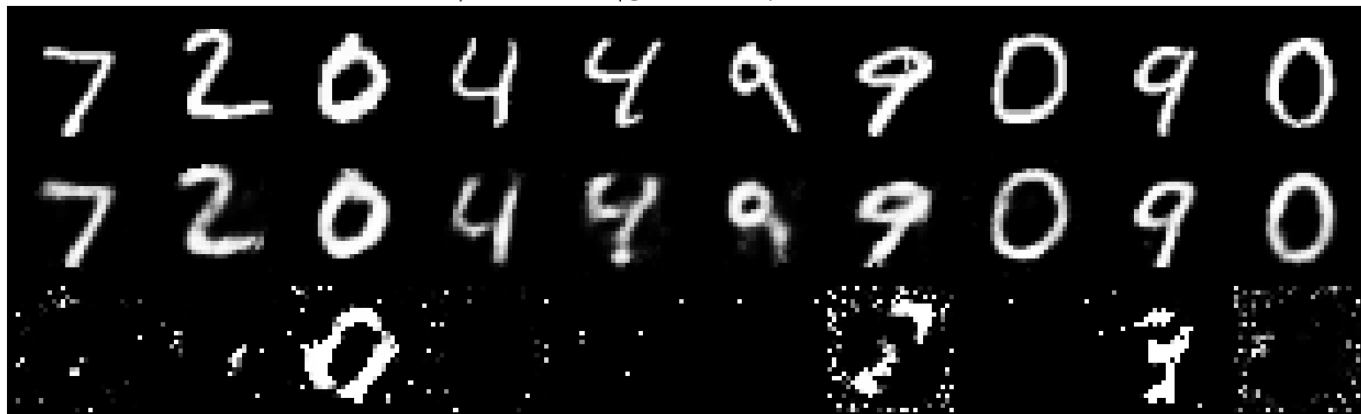


Figure 3: Algorithm 3 saved MNIST output.

Rows: real MNIST | reconstructions | generated samples conditioned on the row labels



Figure 4: Algorithm 4 saved MNIST output.

Rows: real MNIST | reconstructions | generated samples conditioned on the row labels

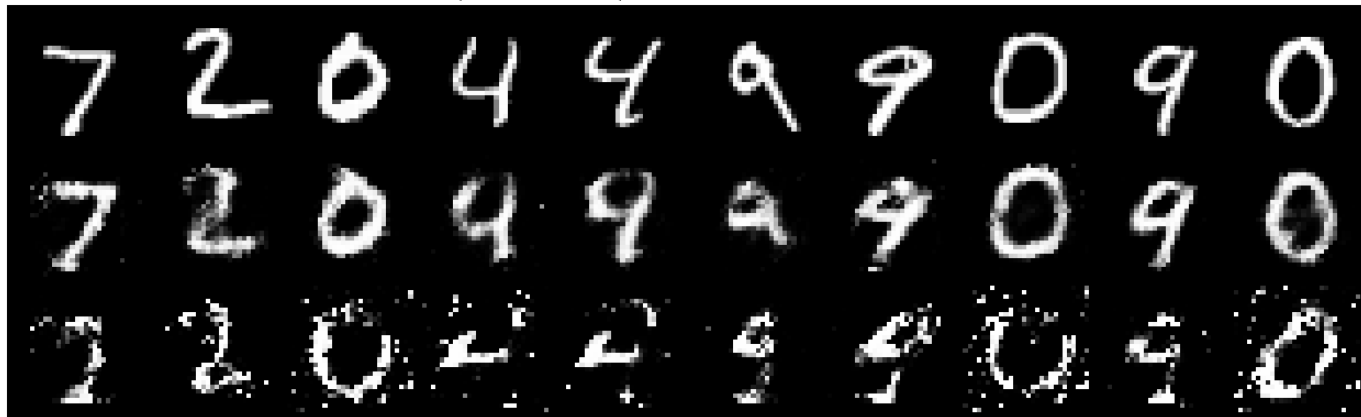


Figure 5: Algorithm 5 saved MNIST output.

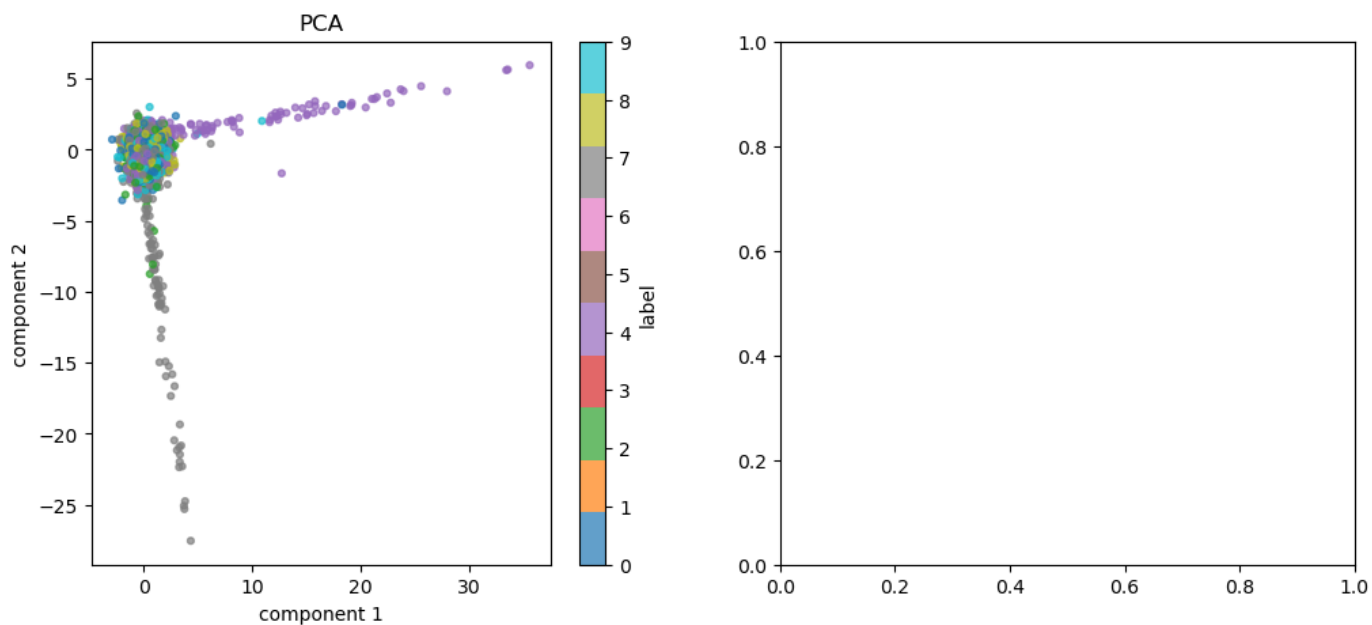


Figure 6: Algorithm 1 PCA/UMAP projection colored by digit.

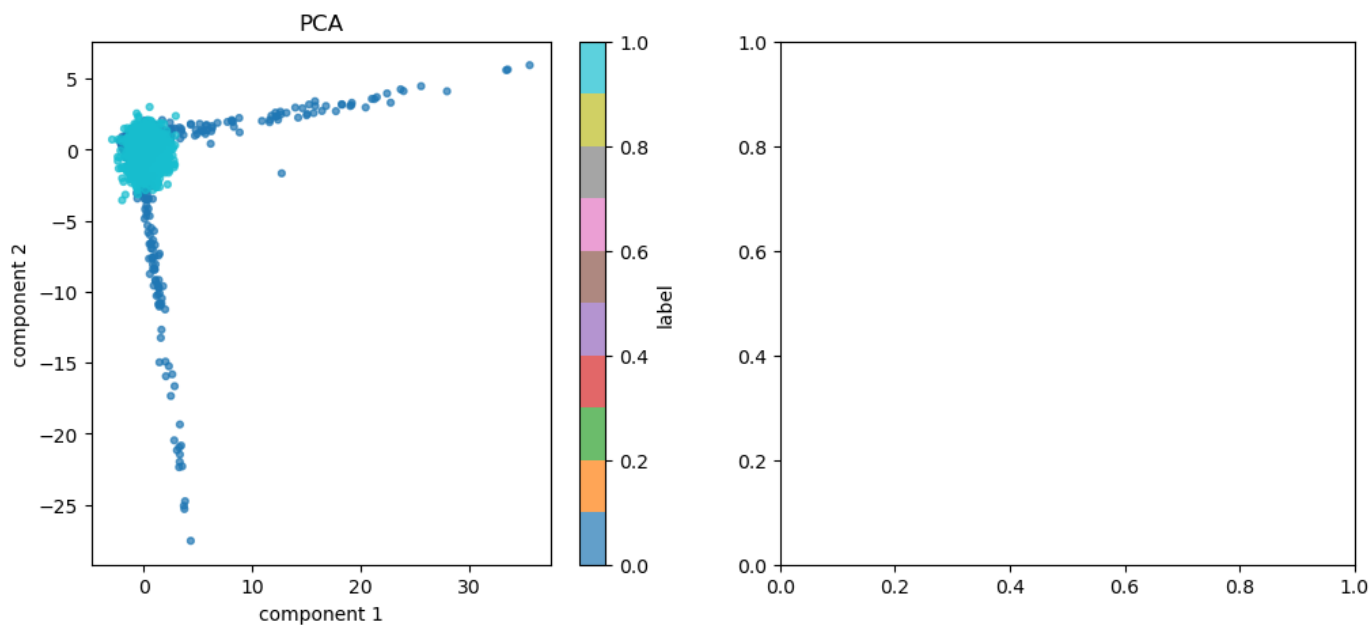


Figure 7: Algorithm 1 PCA/UMAP projection colored by real/generated source.

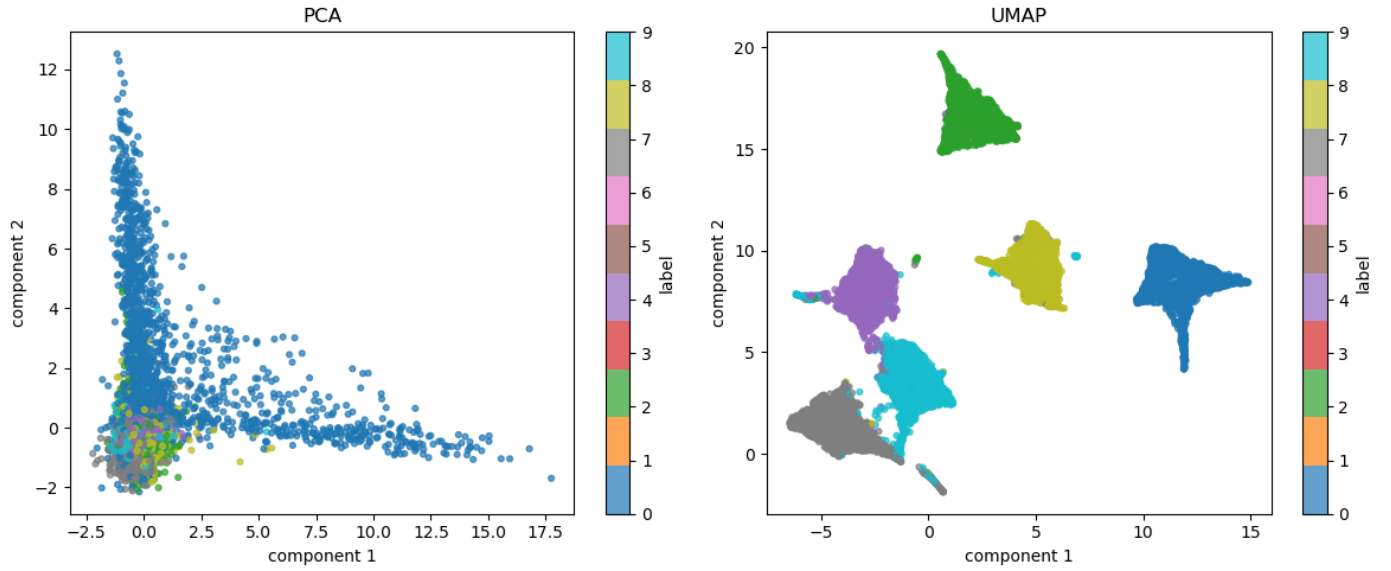


Figure 8: Algorithm 2 PCA/UMAP projection colored by digit.

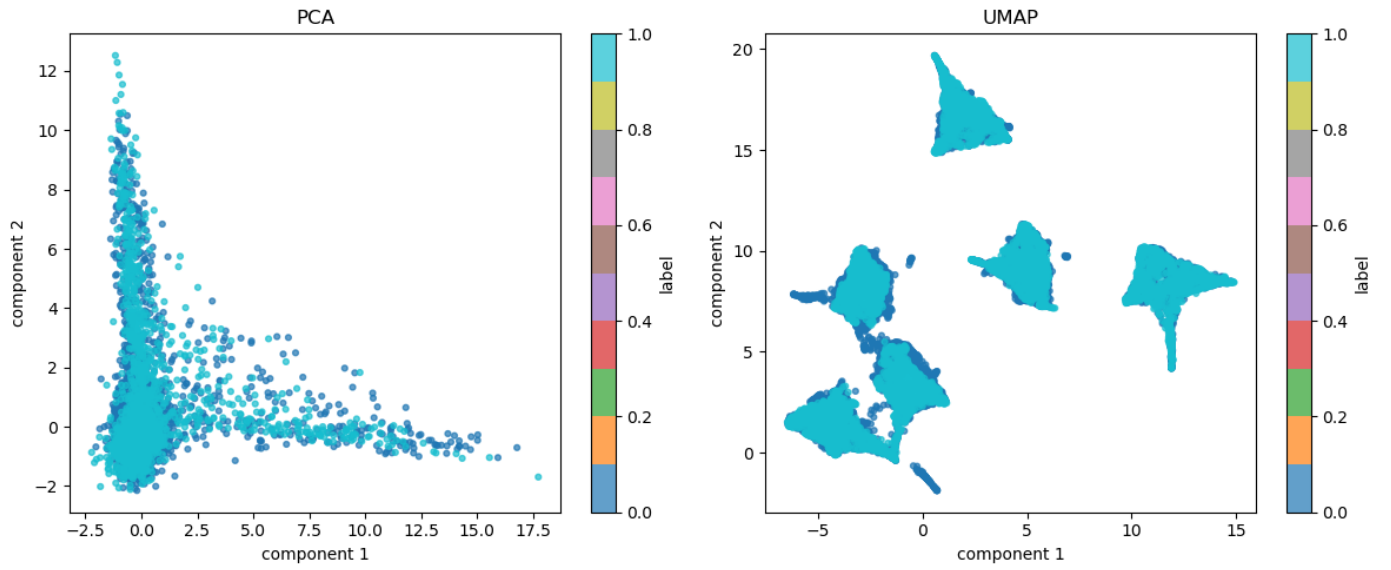


Figure 9: Algorithm 2 PCA/UMAP projection colored by real/generated source.

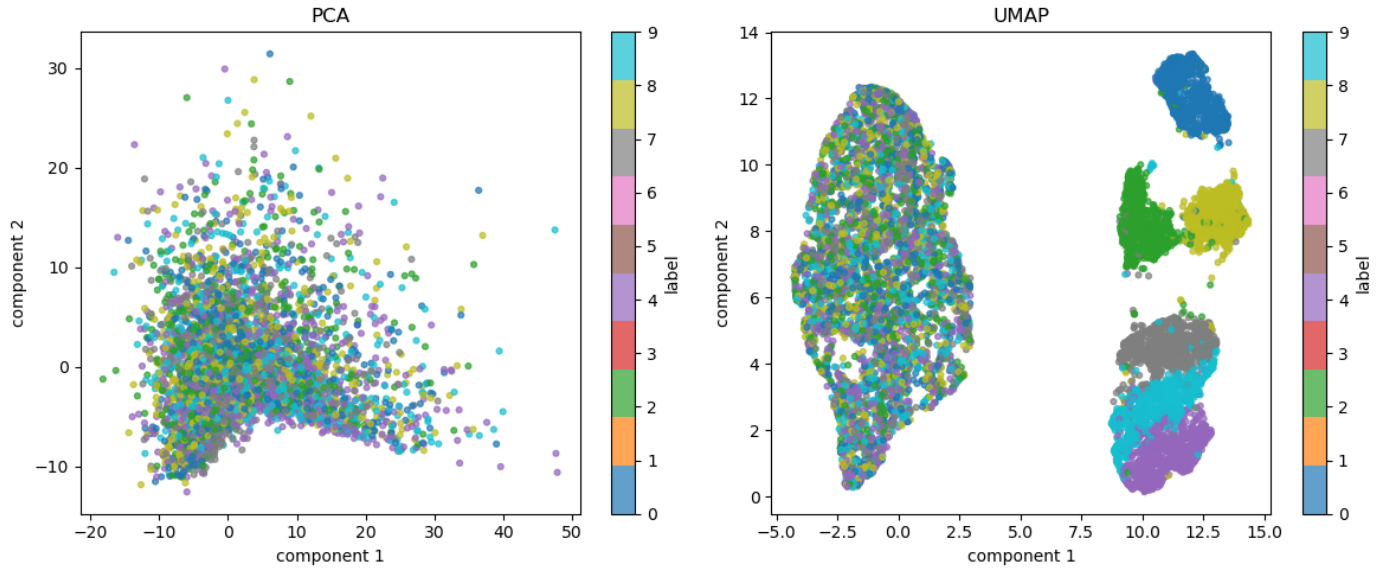


Figure 10: Algorithm 3 PCA/UMAP projection colored by digit.

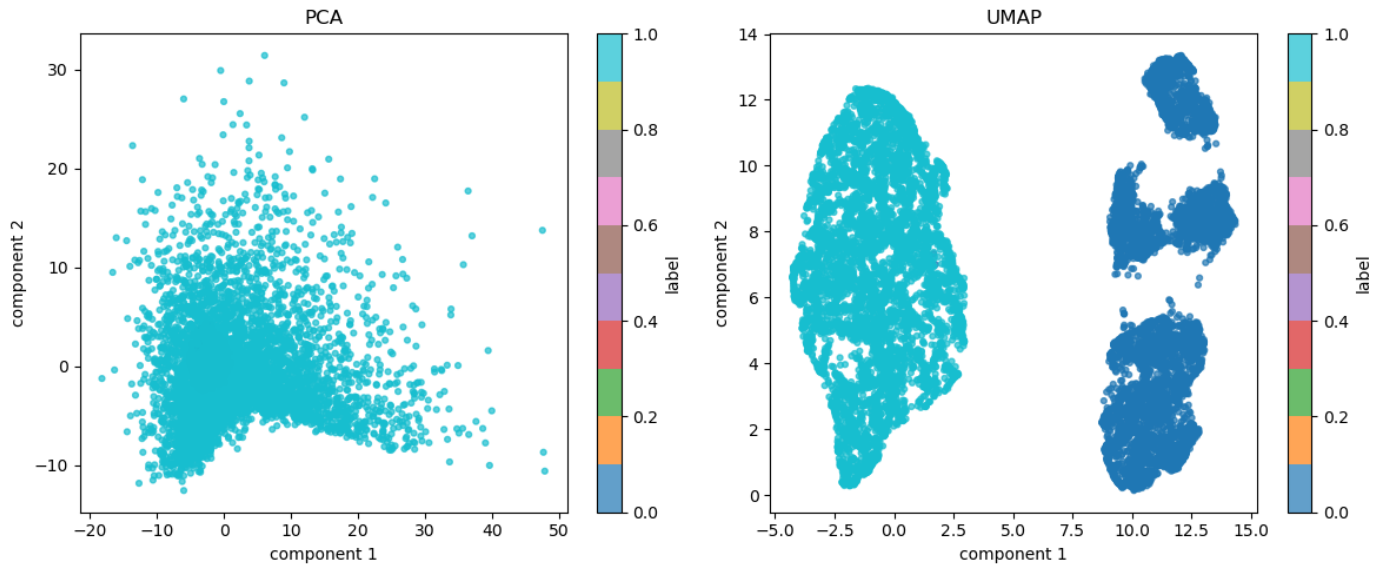


Figure 11: Algorithm 3 PCA/UMAP projection colored by real/generated source.

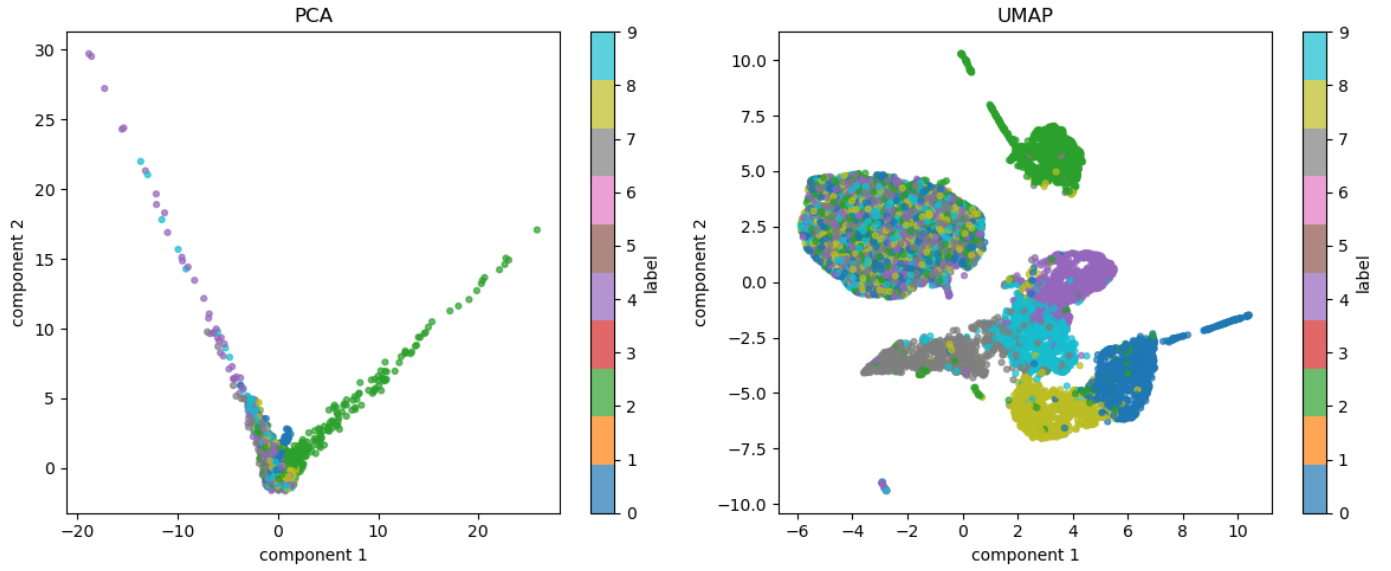


Figure 12: Algorithm 4 PCA/UMAP projection colored by digit.

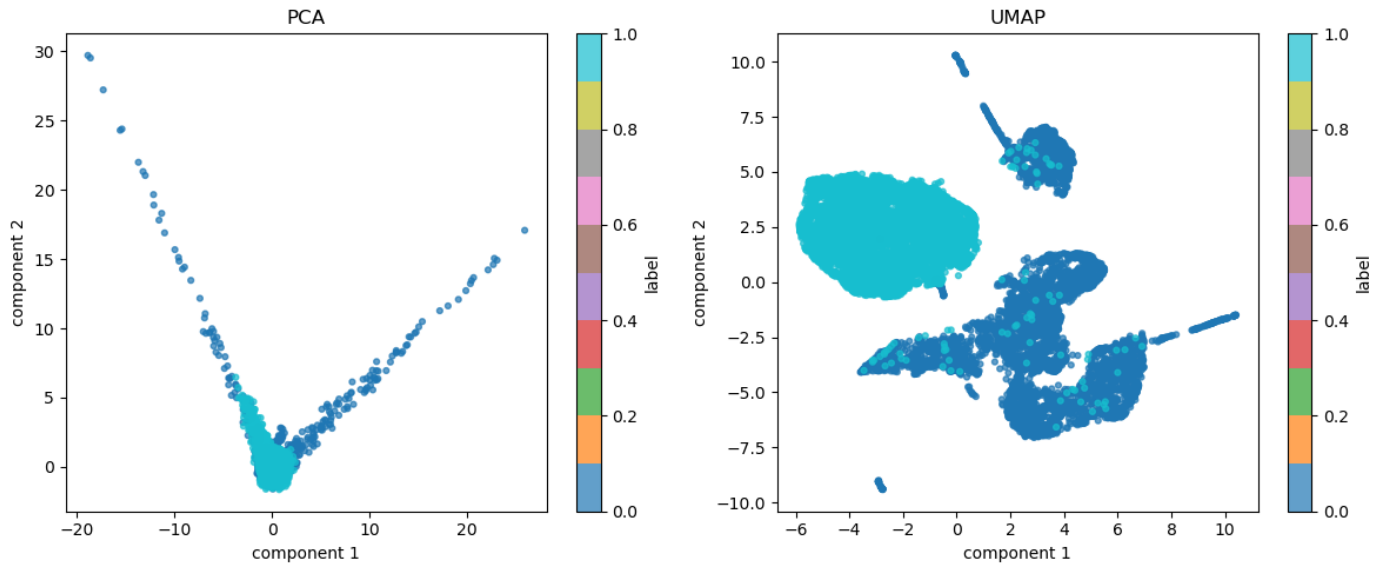


Figure 13: Algorithm 4 PCA/UMAP projection colored by real/generated source.

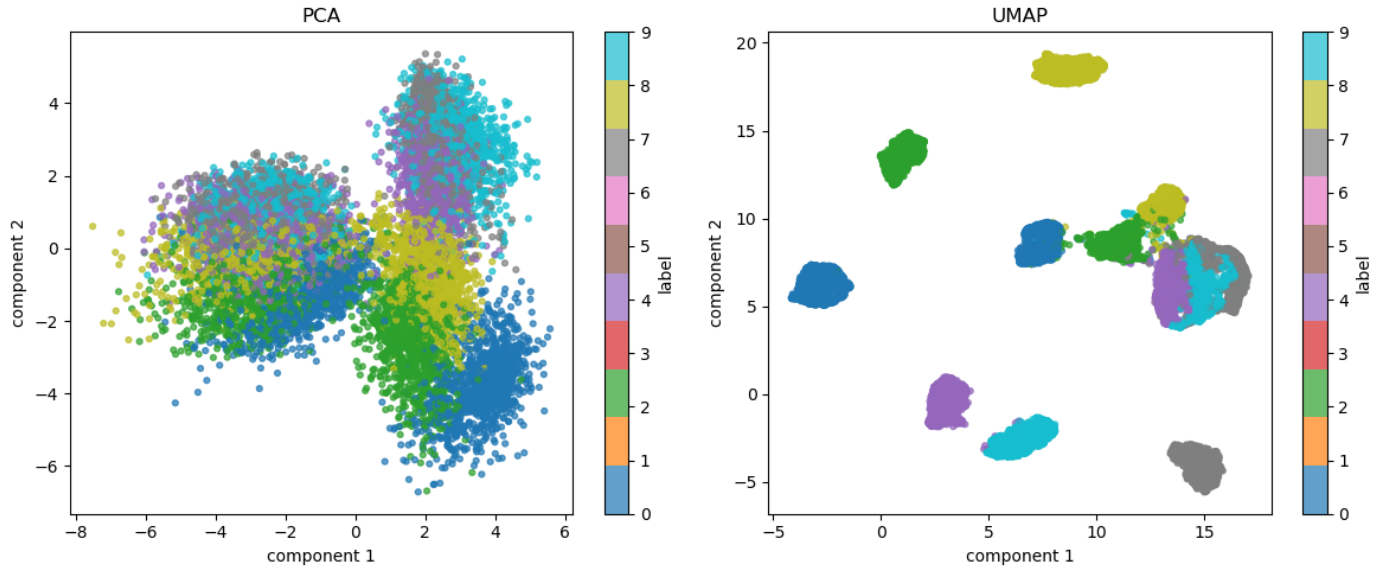


Figure 14: Algorithm 5 PCA/UMAP projection colored by digit.

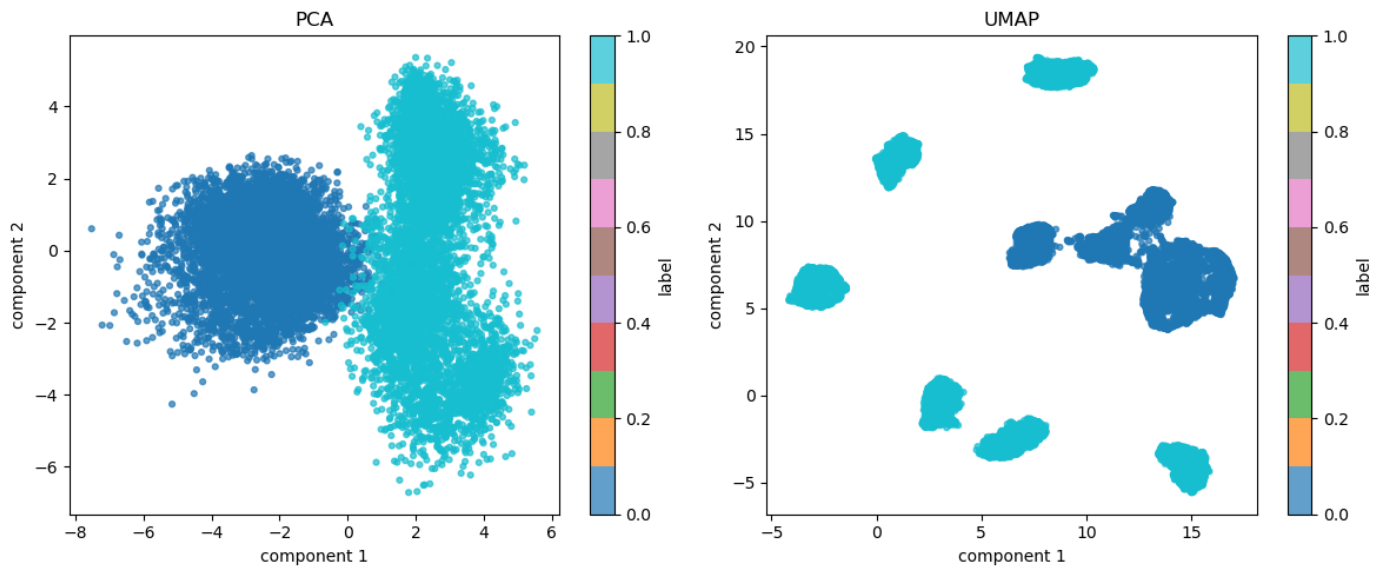


Figure 15: Algorithm 5 PCA/UMAP projection colored by real/generated source.

Gradient-flow diagrams

The current `formulations.tex` contains the combined gradient-flow diagram for Algorithms 1–3. It is included below as page 3 of its compiled PDF. The source file does not currently contain separate Algorithm 4 and Algorithm 5 diagrams, so their flow diagrams are included immediately after it using the computation graphs implemented in their notebooks.

Algorithm 4 flow

Algorithm 5 flow

Algorithm 3 is fully joint: drift is evaluated after decode-re-encode, so its gradient traverses $\tilde{Z}^- \leftarrow D_\psi \leftarrow G_\theta$ and $\tilde{Z}^- \leftarrow \mu_\phi$. No additional cycle loss is used.

Algorithm 3 Algorithm 3 training

Require: $\mathcal{D}$, T , landmark count r , step size η , weights λ

Ensure: (ϕ, ψ, θ)

- 1: Initialize (ϕ, ψ, θ) ; build fixed $\{U_c, W_c, A_c, b_c\}_{c=1}^C$
 - 2: **for** $t = 0, \dots, T_{\text{train}}$ **do**
 - 3: Sample (X, c) and (Ξ, ε)
 - 4: $Z^+ \leftarrow q_\phi(X; \Xi)$; $\hat{X} \leftarrow D_\psi(Z^+)$
 - 5: $Z^- \leftarrow G_\theta(\varepsilon, c)$; $X^- \leftarrow D_\psi(Z^-)$; $\tilde{Z}^- \leftarrow \mu_\phi(X^-)$
 - 6: Compute $V(\text{sg}(\tilde{Z}^-), c)$ and $\mathcal{L}_t$ from (16)
 - 7: $(\phi, \psi, \theta) \leftarrow (\phi, \psi, \theta) - \eta \nabla_{\phi, \psi, \theta} \mathcal{L}_t$
 - 8: **end for**
 - 9: **return** (ϕ, ψ, θ)
-

Loss-to-parameter flow

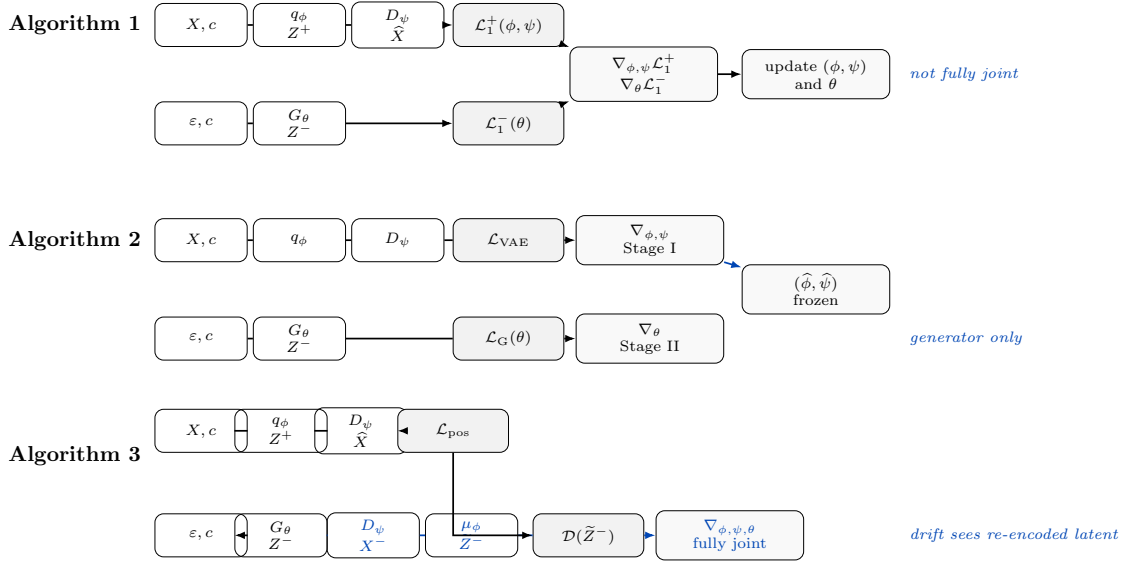


Figure 1: Loss-to-parameter flow. Algorithm 1 has two disconnected gradient blocks despite using one optimizer loop; Algorithm 2 separates VAE and generator stages; Algorithm 3 connects the generated latent to all three parameter blocks through decode-re-encode. Stop-gradient targets do not block gradients through the differentiable first arguments.

Figure 16: Gradient-flow diagram for Algorithms 1–3, included from the compiled `formulations.tex`.

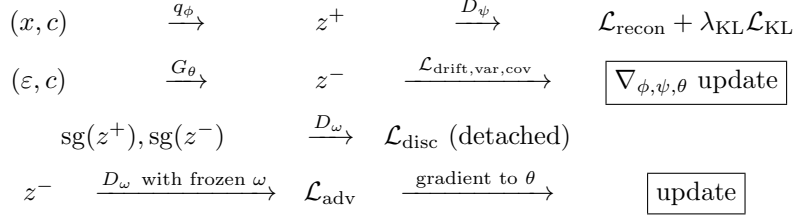


Figure 17: Algorithm 4 adds a detached discriminator-training path and an adversarial gradient path to the generator.

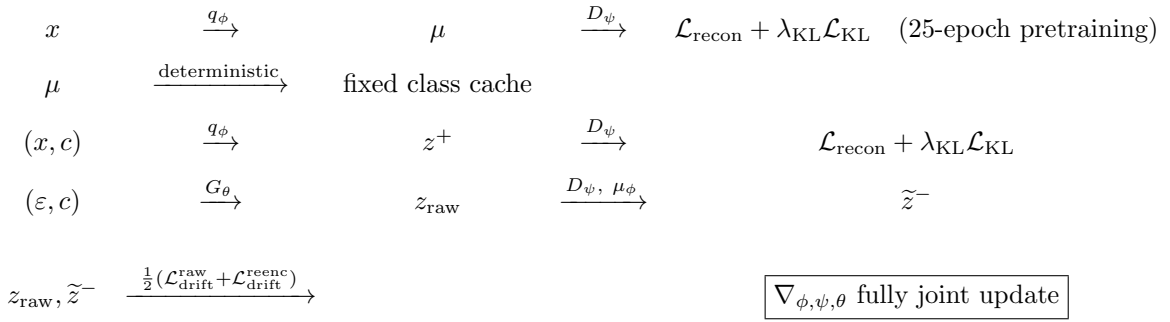


Figure 18: Algorithm 5 first stabilizes the VAE and deterministic cache, then combines raw-latent and re-encoded drift during fully joint training.